

FASE 0 · O AGENTE EM CÓDIGO PURO

Aula 0.6

Saída estruturada e resiliência a erro

Erro de ferramenta vira observação que o agente contorna. E a resposta final vira contrato validado, pronto para outro sistema consumir.

Série: Engenharia de IA Aplicada · Agentes e RAG em Produção

Projeto: Núcleo de Agentes Operacionais

Duração de estudo: cerca de 1h30 · 25 min de teoria e 55 de código

Pré-requisito: Aulas 0.1 a 0.5 (tag **aula-0.5**)

1. Objetivo de aprendizagem

Ao final desta aula o seu agente sobrevive a ferramenta quebrada e a argumento inventado pelo modelo, se recuperando sozinho, e devolve uma resposta final validada contra um contrato, pronta para ser consumida por outro sistema.

2. Por que esta aula importa

Na aula anterior o Núcleo qualificou leads de ponta a ponta. Mas funcionou porque tudo deu certo. Em produção nada dá certo o tempo todo: o banco cai por onze minutos numa manhã de sábado, o modelo chama a atualização sem dizer qual campo, alguém pede uma prioridade "urgentíssima" que não existe no seu enum.

Um agente que morre na primeira exceção não é um produto. É uma demo com sorte.

E tem o outro lado. A resposta final do agente hoje é texto livre. Texto livre é ótimo para humanos e péssimo para sistemas. Quando o Núcleo estiver atrás de uma API na Fase 4, alimentando um CRM real, alguém vai precisar de um objeto com campos, não de um parágrafo simpático.

3. Teoria

3.1 As duas fronteiras que precisam de contrato

Estas duas competências parecem separadas e são a mesma ideia aplicada em dois lugares: **onde o texto do modelo encontra o seu código, precisa haver validação.**

A primeira fronteira é a **entrada da ferramenta**. O modelo gera os argumentos, e ele pode gerar qualquer coisa: campo faltando, tipo errado, valor fora do enum. Validar aqui protege o seu banco.

A segunda é a **saída final do agente**. Se outro sistema vai consumir, o formato precisa ser garantido, não esperado.

Nos dois casos o instrumento é o mesmo: Pydantic. Você declara o contrato como classe, e a validação vem de graça, com mensagem de erro legível.

3.2 Erro de ferramenta é observação, não exceção

Este é o insight central da aula, e ele é contraintuitivo.

Quando uma ferramenta falha, o instinto do programador é deixar a exceção subir. Num agente, isso é quase sempre errado. A exceção mata o laço, e o agente perde a chance de contornar.

ANALOGIA

Pense em como um humano lidaria. Você tenta agendar uma reunião, o sistema diz que o horário está ocupado, e você propõe outro. Não abandona a tarefa nem sai da sala. O agente pode fazer exatamente isso, **desde que o erro chegue até ele como informação.**

Então a regra é: capture a exceção da ferramenta e devolva a mensagem de erro como resultado, no lugar do resultado bem-sucedido. O laço continua, o modelo lê "prioridade deve ser baixa, media ou alta" e corrige na volta seguinte.

Repare que isso é a regra 4 da aula passada levada ao limite: o retorno da ferramenta é texto que o modelo lê, inclusive quando é um erro. Um erro que não diz o que corrigir faz o modelo repetir a mesma chamada e entrar em ciclo.

3.3 Onde a regra se inverte

Nem todo erro deve virar observação. Falha de autenticação, provider fora do ar, banco inacessível: isso é infraestrutura, o agente não tem como contornar, e insistir só queima token.

A distinção é sua, não do framework, e cabe numa pergunta: **o agente consegue fazer algo diferente da próxima vez?** Se sim, é observação. Se não, levante e saia. Por isso criamos uma exceção própria, para marcar explicitamente o que é irrecuperável.

3.4 Validar não é confiar

Uma advertência que vale para toda a Fase 3. Validação garante **formato**, nunca **verdade**. Um contrato perfeitamente válido pode conter uma pontuação inventada e uma justificativa fantasiosa. O Pydantic não tem como saber.

Formato é problema de engenharia e se resolve aqui. Verdade é problema de avaliação e se resolve com instrumento próprio, na Fase 3. Confundir os dois é como achar que um formulário bem preenchido é um formulário honesto.

3.5 Uma decisão de projeto sobre saída estruturada

Os dois providers têm mecanismos **nativos** de saída estruturada, com nomes, garantias e assinaturas diferentes, e que mudam com frequência. Em vez de amarrar o Núcleo a eles, implementamos o padrão portátil: pedir o JSON pelo prompt, validar com Pydantic e, se falhar, **devolver o erro de validação ao modelo para ele corrigir**.

O que isso nos dá: funciona igual nos dois providers, usa a `chat()` da aula 0.1 sem nenhuma mudança, e é testável sem rede. O que custa: uma chamada a mais quando o modelo erra o formato. Quando você for para produção com um provider só, vale conferir o mecanismo nativo dele na documentação e trocar esse arquivo. O resto do Núcleo não muda.

E repare na simetria: o ciclo de correção é o mesmo princípio da resiliência de ferramenta. Falha vira informação, não exceção.

4. Mão na massa

Três arquivos novos e nenhuma alteração nos anteriores. O `agente.py` segue intocado desde a aula 0.3, e você vai ver como.

Passo 1 — O contrato de saída

O que e por quê: declarar o formato esperado como classe. As regras entram no contrato, não num if espalhado pelo código.

```
class QualificacaoLead(BaseModel):
    """O que o Nucleo devolve para outro sistema consumir."""

    email: str = Field(description="Email do lead qualificado")

    status: Literal["qualificado", "nutrir", "descartado"] = Field(
        description="Decisao final sobre o lead")
```

```

pontuacao: int = Field(ge=0, le=100,
                      description="Score de 0 a 100")

justificativa: str = Field(
    description="Por que este status, em uma frase")

proxima_acao: str = Field(
    description="O proximo passo concreto, em uma frase")

# Literal fecha o vocabulario do status; ge e le impedem pontuacao 150.
# O contrato carrega as REGRAS, nao so os nomes dos campos.

```

O que faz: `Literal` impede um status inventado, `ge` e `le` impedem pontuação 150. O `description` de cada campo não é enfeite: ele vira o JSON Schema que pedimos ao modelo no Passo 4.

Passo 2 — A fronteira do irrecuperável

O que e por quê: uma exceção própria para marcar o que o agente não contorna. Sem ela, todo erro viraria observação e um banco fora do ar geraria seis tentativas inúteis.

```

class ErroIrrecuperavel(Exception):
    """Falha que o agente nao tem como contornar.

    Provider fora do ar, credencial expirada, banco inacessivel.
    Insistir so queima token: deixamos subir e o laco para.
    """

def _mensagem_de_validacao(erro):
    """Transforma o erro do Pydantic em instrucao para o modelo.

    Nao basta dizer que falhou: o texto precisa dizer QUAL campo e
    POR QUE, senao o modelo tenta de novo do mesmo jeito.
    """
    partes = []
    for e in erro.errors():
        campo = ".".join(str(x) for x in e["loc"]) or "(argumento)"
        partes.append(f"{campo}: {e['msg']}")
    return "; ".join(partes)

```

Passo 3 — Proteger a ferramenta

O que e por quê: o coração da aula. Um wrapper que valida os argumentos e transforma falha em texto.

```

def proteger(func):
    """Embrulha uma ferramenta para que ela nunca derrube o agente.

    Faz duas coisas:
    1. valida os argumentos gerados pelo modelo contra os type hints
       da propria funcao (inclusive Literal, que virou enum na 0.5);
    2. captura a falha de execucao e devolve texto que o agente le.

    O wrapper aceita **kwargs em vez da assinatura real de proposito:
    se o modelo inventar um argumento que nao existe, queremos tratar
    isso como observacao, nao como TypeError do Python.
    """

```

```

validada = validate_call(func) # valida contra os type hints

def executor_seguro(**kwargs):
    try:
        return validada(**kwargs)

    except ValidationError as e:
        # O modelo errou ou inventou um argumento. Devolvemos o
        # diagnostico para ele corrigir na proxima volta do laço.
        return (f"ERRO DE ARGUMENTO em {func.__name__}: "
                f"{_mensagem_de_validacao(e)}. "
                "Corrija os argumentos e chame a ferramenta de novo.")

    except TypeError as e:
        # Argumento a mais, a menos, ou com nome errado.
        return (f"ERRO DE ARGUMENTO em {func.__name__}: {e}. "
                "Confira os parametros no schema da ferramenta.")

    except ErroIrrecuperavel:
        # Infraestrutura: o agente nao contorna. Deixa subir.
        raise

    except Exception as e:
        # A ferramenta quebrou por conta propria. Tambem e
        # observacao: o agente pode tentar outro caminho.
        return (f"ERRO NA FERRAMENTA {func.__name__}: "
                f"{type(e).__name__}: {e}")

executor_seguro.__name__ = func.__name__
return executor_seguro

def executores_seguros():
    """Igual ao executores() da aula 0.4, porem com tudo protegido.

    E o que permite o laço da aula 0.3 continuar intocado: ele recebe
    um dicionario de funcoes e nao precisa saber que elas agora se
    defendem sozinhas.
    """
    return {nome: proteger(f.funcao) for nome, f in REGISTRO.items()}

```

O que faz: `validate_call` do Pydantic valida contra os *type hints* da própria função, inclusive o `Literal` que virou enum na aula passada. Fonte única da verdade de novo: não escrevemos um validador separado.

Repare no `**kwargs` do wrapper, que é deliberado: se o modelo inventar um argumento que não existe, queremos tratar como observação e não como `TypeError` do Python estourando antes de o nosso código rodar.

E repare no `executores_seguros()`: ele devolve exatamente o mesmo formato que `rodar_agente` já esperava. **Por isso o laço não muda.** Trocar `executores()` por `executores_seguros()` no rascunho é a diferença entre um agente frágil e um resiliente.

Passo 4 — A saída estruturada com ciclo de correção

O que e por quê: converter a resposta em linguagem natural num objeto validado, com o erro voltando ao modelo quando ele erra o formato.

```

def extrair_estruturado(texto_bruto, modelo_pydantic, provider="anthropic",
                        tentativas=2):
    """Converte a resposta final do agente num objeto validado.

```

```

texto_bruto -> a resposta em linguagem natural do agente
modelo_pydantic -> a classe do contrato (ex.: QualificacaoLead)

Se a validacao falhar, o ERRO volta para o modelo como instrucao de
correcao. E o mesmo principio da resiliencia de ferramenta: falha
vira informacao, nao execucao.
"""

mensagens = [{
    "role": "user",
    "content": ("Extraia os dados estruturados do relato abaixo.\n\n"
               + texto_bruto),
}]

ultimo_erro = None

for _ in range(tentativas):
    resposta = chat(provider, mensagens,
                    system=_instrucao(modelo_pydantic))
    bruto = _limpar(resposta)

    try:
        return modelo_pydantic.model_validate_json(bruto)

    except ValidationError as e:
        ultimo_erro = e
        # Devolve ao modelo o que ele produziu e o que esta errado.
        mensagens.append({"role": "assistant", "content": bruto})
        mensagens.append({
            "role": "user",
            "content": ("O JSON acima nao passou na validacao:\n"
                       f"{e}\n\nCorrija e responda apenas com o "
                       "JSON valido."),
        })

    except (json.JSONDecodeError, ValueError) as e:
        ultimo_erro = e
        mensagens.append({"role": "assistant", "content": bruto})
        mensagens.append({
            "role": "user",
            "content": (f"Isso nao e JSON valido ({e}). Responda "
                       "apenas com o objeto JSON, sem markdown."),
        })

```

O que faz: a instrução de sistema é montada a partir do `model_json_schema()` do próprio Pydantic, então o contrato segue sendo a fonte única. Se a validação falhar, acrescentamos ao histórico o que o modelo produziu e o que está errado, e pedimos de novo. Esgotadas as tentativas, levantamos: quem chamou pediu um contrato e não recebeu, e devolver um objeto meia-boca seria pior do que falhar.

Passo 5 — Ver funcionando

O que e por quê: o mesmo fluxo da aula anterior, com uma linha de import trocada e a extração no final.

```

# scratch/rascunho_0_6.py (descartavel)
from nucleo import banco, ferramentas
from nucleo.registro import schemas
from nucleo.resiliencia import executores_seguros    # <- a unica mudanca
from nucleo.estruturado import extrair_estruturado
from nucleo.modelos import QualificacaoLead

```

```

from nucleo.agente import rodar_agente

banco.criar_schema()

SISTEMA = """Voce e o assistente comercial do FlowDesk, um SaaS B2B.
Consulte o CRM antes de agir. Lead com plano pro ou enterprise, ou MRR
acima de 300, e qualificado: atualize o status e proponha reuniao.
Problema tecnico vira ticket, nunca reuniao."""

exe = rodar_agente(
    "A Ana do TechFlow relatou erro 500 no login e quer migrar de plano.",
    schemas(), executores_seguros(), system=SISTEMA)

print(exe.resposta)

# A resposta em linguagem natural vira contrato para outro sistema.
dados = extrair_estruturado(exe.resposta, QualificacaoLead)
print(dados.model_dump_json(indent=2))

```

Resultado esperado: o agente resolve o caso e você recebe um JSON validado. Agora o teste de verdade: edite a ferramenta de ticket para levantar uma exceção de propósito, ou peça ao modelo uma prioridade que não existe. O agente lê o erro, corrige e segue. Antes desta aula, ele morria.

Passo 6 — Os testes

O que e por quê: dez testes determinísticos. Dois merecem destaque. O primeiro prova que a regra tem limite:

```

# ----- 5. erro irrecuperavel SOBE, nao vira observacao -----

def test_erro_irrecuperavel_sobe():
    def ferramenta_sem_credencial(x: int) -> str:
        """Simula credencial expirada."""
        raise ErroIrrecuperavel("chave de API expirada")

    with pytest.raises(ErroIrrecuperavel):
        proteger(ferramenta_sem_credencial)(x=1)
    # 0 agente nao contorna isso. Insistir so queimaria token.

```

E o segundo prova o ciclo de correção, sem chamar API nenhuma:

```

# ----- 9. modelo erra, recebe o erro e corrige -----

def test_corrige_na_segunda_tentativa(monkeypatch):
    respostas = [
        # Primeira: pontuacao fora da faixa e cercado de markdown.
        ```json\n{"email": "c@d.io", "status": "qualificado", "
 "pontuacao": 150, "justificativa": "x", "proxima_acao": "y"}\n```
 VALIDO,
]
 enviadas = []

 def fake_chat(provider, mensagens, system=None, max_tokens=1024):
 enviadas.append(list(mensagens))
 return respostas.pop(0)

 monkeypatch.setattr(estruturado, "chat", fake_chat)
 obj = estruturado.extrair_estruturado("relato", QualificacaoLead)

 assert obj.pontuacao == 85

```

```
Na segunda chamada, o historico levou o erro de volta ao modelo.
texto_segunda = str(enviadas[1])
assert "nao passou na validacao" in texto_segunda
```

**Resultado esperado:** `pytest` mostra `43 passed, 2 deselected`.

## Passo 7 — Commit e tag

```
pytest -v # esperado: 43 passed, 2 deselected
git status # .env, scratch/ e *.db NAO podem aparecer
git add .
git commit -m "Aula 0.6: saida estruturada e resiliencia a erro"
git tag aula-0.6
git push && git push --tags
```

## 5. Portabilidade

Erro como observação é o padrão em todo framework sério de agente. No LangGraph, o `ToolNode` tem um parâmetro para tratamento de erro que faz exatamente isto: devolve a mensagem como resultado da ferramenta em vez de propagar a exceção. Agora você sabe por que ele existe e o que escolher quando a opção aparecer.

O ciclo "valida, falhou, devolve o erro ao modelo, tenta de novo" aparece com nomes diferentes em todo lugar: *retry with repair*, *reflection on error*, *self-healing output*. É o mesmo mecanismo.

E a distinção entre recuperável e irrecuperável não é de IA: é o velho princípio de tratar exceções no nível em que elas podem ser resolvidas.

## 6. Terminei quando

O critério da aula: **eu quebro uma ferramenta de propósito e o agente contorna sem cair**.

- Uma prioridade fora do enum vira mensagem de erro legível, e o agente corrige na volta seguinte.
- Uma ferramenta que levanta exceção não derruba o laço.
- Um erro marcado como irrecuperável **derruba**, e você sabe explicar por que a diferença existe.
- A resposta final sai como objeto validado, e um JSON inválido do modelo é corrigido numa segunda tentativa.
- `pytest` mostra `43 passed, 2 deselected`.

## 7. Erros comuns e troubleshooting

- **O agente entra em ciclo depois de um erro.** A mensagem não diz o que corrigir. "Erro de validação" sozinho faz o modelo repetir a mesma chamada; ele precisa saber qual campo e por quê.
- **Tudo virou observação, inclusive credencial expirada.** Faltou usar a exceção de irrecuperável nas falhas de infraestrutura. O sintoma é o agente dando seis voltas e queimando token sem chance de sucesso.

- **TypeError estourando antes do wrapper.** O wrapper precisa aceitar `**kwargs`. Com a assinatura real, Python rejeita o argumento inventado antes de o seu código rodar.
- **O modelo devolve JSON cercado de crases.** É comum, e a limpeza de markdown resolve. Se persistir, reforce na instrução que não deve haver texto antes nem depois.
- **A extração falha sempre, mesmo com retry.** Normalmente o contrato está pedindo um campo que a resposta do agente simplesmente não contém. Validação não inventa dado: revise o contrato ou o prompt do agente.
- **Um objeto válido com dados errados.** É o esperado, e é o assunto da Fase 3. Formato não é verdade.

## 8. Ponte para a próxima aula

---

O Núcleo agora é resiliente, mas você só sabe o que aconteceu olhando o terminal enquanto ele roda. Na **Aula 0.7, Observabilidade desde o berço**, cada execução vira uma árvore rastreada, com custo em tokens e latência por passo. É o que permite responder, numa segunda-feira, por que o agente fez o que fez no sábado.